ePolicies AI Assistant – Limitations and How to Solve Them

Plan for every open limitation found while testing 3 real policies (Abortion, Accident, Advance Directives) offline and with real Gemini answers. Rate limits and overall answer speed are out of scope for this plan. Nothing in this document has been implemented yet.

Summary

#	Limitation	Solution in one line	Effort	Re-index
1	Referenced policies followed too broadly	Follow only references related to the question / missing info	Small	No
2	Out-of-scope questions slow and costly	Stop early when nothing relevant is found; fewer extra calls	Small	No
3	Common words trigger a spelling-correction call	Treat a word as a typo only if it is close to a real policy word	Small	No
4	Keyword search misses some formats	Split slash words, expand code ranges, match codes/dates/short forms	Medium	Yes
5	Small sections lose to long ones	Boost heading matches in keyword ranking	Small	No
6	Answers limited to indexed policies	Tell the user which referenced policy to open; check missing links	Small	No
7	URLs are 17% of the indexed text	Embed link text only; keep URLs for source cards	Small	Yes
8	Collection questions have gaps	Accept typos, "mention/in X", search section names too	Small	No
9	Source data quality	Clean known scraper problems at load time; show effective dates	Medium	Yes
10	Not tested on all 235 policies / no accuracy numbers	30-question test set + `evaluate` before and after	Medium	No

All items with "Re-index: Yes" can be released together, so the laptop needs only **one** `python -m app.cli ingest`; unchanged text reuses cached embeddings.

1. Referenced policies are followed too broadly

Problem: when evidence is incomplete, every "See: [Policy]" link in the passages is searched, even unrelated ones.

Example (real test): "Which code is used for an induced abortion at 20 weeks?" also searched *Maternity – Length of Hospital Stay*, *Nonparticipating Provider Authorizations*, *ASH Precertification* and *Legislation by Topic* → 2 extra rounds, about 7,000 extra tokens.

Solution:

1. Follow a reference only when its name shares a key word with the **question** or with the evidence checker's **missing information** (for example "precertification", "travel").
2. Follow at most **2** references per round, most relevant first; never the same one twice ("Legislation by Topic" was listed twice).
3. Skip references to sections of a document that is already in the context.

Where: `backend/app/agents/retrieval_agent.py` (`referenced_policies`, retrieval loop).

Expected result: the 20-weeks question finishes in 1 round instead of 3; questions that really need another policy ("travel" → *Travel and Lodging – Procedures*) still follow it.

Test: the 20-weeks and travel questions; count rounds and tokens in Diagnostics.

2. Out-of-scope questions are slow and expensive

Problem: "What is the office parking policy?" took 72 s and 5 Gemini calls before answering "not found".

Solution:

1. After round 1, if no passage has a meaningful match (low vector similarity **and** no keyword hit on the question's main words), stop and answer "not found" without more rounds.
2. Do not follow referenced policies (item 1) when the evidence checker marked **no** passage as relevant.
3. Combined with item 3, the spelling-correction call is skipped for normal words like "parking".

Where: `retrieval_agent.py` (stop rule), `rag_service.py`.

Expected result: out-of-scope questions in about 1–2 Gemini calls instead of 5.

Test: "office parking policy", "cafeteria hours", "IT password reset" (not in the 3 policies).

3. Common words trigger an unnecessary spelling-correction call

Problem: any word found in no policy is treated as a typo, so normal words cost an extra Gemini call: "everything", "parking", "kind", "dept", "RU-486".

Solution:

1. Only treat a word as a typo when it is **close to a real policy word** (one or two letters different: "subrogaton" → "subrogation", "jpoint" → "joint").
2. Ignore common English words, numbers and number fragments ("486").
3. Keep the note in Diagnostics so it stays visible when it does happen.

Where: `backend/app/retrieval/keyword_search.py` (`unknown_terms`), `rag_service.py`.

Expected result: "jpoint partership" and "subrogaton for acident" are still corrected; "tell me everything…" and "parking" no longer cost an extra call.

Test: the typo questions from the earlier screenshots plus normal questions with rare words.

4. Keyword search misses some formats

Format	Example	Solution
Slash-joined words	"Rawlings/Machinify", "OB/GYN", "member/patient"	Also index the separate words ("rawlings", "machinify")
Code ranges	99203 inside "99202 - 99205"	Add every code of a range (up to a sensible size) to the index
5-digit procedure codes	"59866"	Treat 5-digit codes as exact identifiers, like "S0190"
Dates	"effective 06/03/25"	Treat dates as exact terms
Hyphen variants	"RU-486" vs "RU486"	Match with and without the hyphen
Short forms	"precert" vs "precertification"	Add to the built-in synonyms (also dept ↔ department)

Where: `keyword_search.py`, `query_planner.py` (identifiers), `synonyms.py`, `repository.py` (search text). **Re-index:** yes.

Expected result: "what does Rawlings Machinify do", "code 99203", "59866" and "precert for stillbirth" find the right section by keyword too, not only by meaning.

5. Small sections lose to long sections in keyword ranking

Problem: the Travel section (65 tokens) ranks below long sections that mention "abortion" many times.

Solution:

1. Give an extra boost when the question's words match the **section heading** (the heading is already stored separately).
2. Optionally reduce the advantage of very long chunks (length-normalised ranking).

Where: `keyword_search.py`.

Expected result: "travel reimbursement for elective abortion" puts *Reimbursement for Travel for an Elective Abortion* first.

6. Answers are limited to the indexed policies

Problem: some sections only point elsewhere ("**See:** Travel and Lodging – Procedures"). If that page is not indexed, the answer can only say "not specified"; links to company websites or Word files can never be answered.

Solution:

1. When the answer is incomplete because of such a link, say so clearly and show the link: *"The Abortion policy refers to Travel and Lodging – Procedures for details; open it here: …"*.
2. Add a report command that lists referenced policies **not** present in the index, so missing pages can be added to the scraper.
3. Keep refusing to guess (strict grounding stays on).

Where: `answer_generator.py` / prompts, new CLI report in `cli.py`.

7. URLs are 17% of the indexed text

Problem: long links (`https://www20.aetna.com/nco/claim_call/…`) are embedded with the text: extra embedding cost and noise in meaning search.

Solution: embed the **link text** only ("Travel and Lodging – Procedures"); keep the full URLs in the chunk's metadata so source cards still show them.

Where: `chunker.py` (`build_embedding_input`). **Re-index:** yes.

Expected result: smaller embedding input and cleaner meaning search; source cards unchanged.

8. Collection questions have gaps

Question	Today	Solution
"how **mny** policies are there"	not recognised	Accept small typos in the key words
"which policies **mention** HRP"	not recognised	Recognise "mention / about / related to / in"
"how many policies are there **in medicare**"	not recognised	Treat "in X" as a filter
"list all policies about **subrogation**"	no title matches	Also search **section headings**, and say which section matched

Where: `backend/app/agents/catalog.py`, `repository.py`.

9. Source data quality (from the scraper)

Problem	Example	Solution
Q&A labels swapped	"**Answer:** How can a member execute an Advance Directive?"	Fix in the scraper; until then, clean known patterns at load time
Formatting leftovers	`**Note****:**`, ` ` in table cells	Clean at load time (`**Note:**`, line breaks)
Effective dates not visible	"Modified – Effective 05/23/23"	Store the section's effective date and show it on the source card
Old sections	"Effective 02/09/15"	Show the date so users can judge how current it is

Where: `markdown_loader.py` / `markdown_parser.py` (cleaning, dates), source card in the website. **Re-index:** yes.

10. Not tested on all 235 policies / no accuracy numbers

Problem: only 3 policies were tested here; there are no accuracy numbers yet.

Solution:

1. Write **30 real questions** from the policies, each with the correct policy file (and section), in `backend/eval/epolicies_eval.jsonl` (the template exists). Include: direct facts, codes, step tables, follow-ups, typos, out-of-scope.
2. Run on the laptop: `python -m app.cli evaluate --with-answers --judge` → Recall@5, MRR, citation precision/recall, faithfulness, answer relevance.
3. Run it **before and after** each batch of fixes to prove the improvement.
4. Report the 235-policy checks: how many documents have deep step lists, acronyms learned, referenced policies missing from the index.

Suggested order

Step	Items	Why first
1	10 (test set, first run)	Gives a baseline to measure every fix against
2	1, 2, 3	Small, no re-index, fewer wasted Gemini calls
3	4, 7, 9	Need a re-index – release together, one `ingest`
4	5, 6, 8	Search ranking and user-facing improvements
5	10 (second run)	Before/after numbers for the manager

Out of scope for this plan

- **Gemini rate limits** – request pacing and higher Vertex AI quota.
- **Answer speed** – fewer Gemini calls per question, streaming, caching.